ARCHITECTURE NOTE · 04

Glassbox agents

The platform blueprint, the reference implementation that proves it runs, and the stack I actually build and ship on today.

The three tools in the other notes are each one service line's answer. This is the general shape underneath them: a **two-lane platform** — a sovereign local lane for confidential client work, a cloud lane for everything else, sharing one data layer, one orchestration standard and one governance plane.

The argument for that shape is not technical. Under the EU AI Act, NIS2 and DORA, the defensible differentiator is not agent capability; it is being able to show a regulator, a client and your own quality function exactly what an agent saw, decided and did. Governance is the product, not the overhead. glassbox-agents is that blueprint as working, tested Python — zero third-party dependencies in the core.

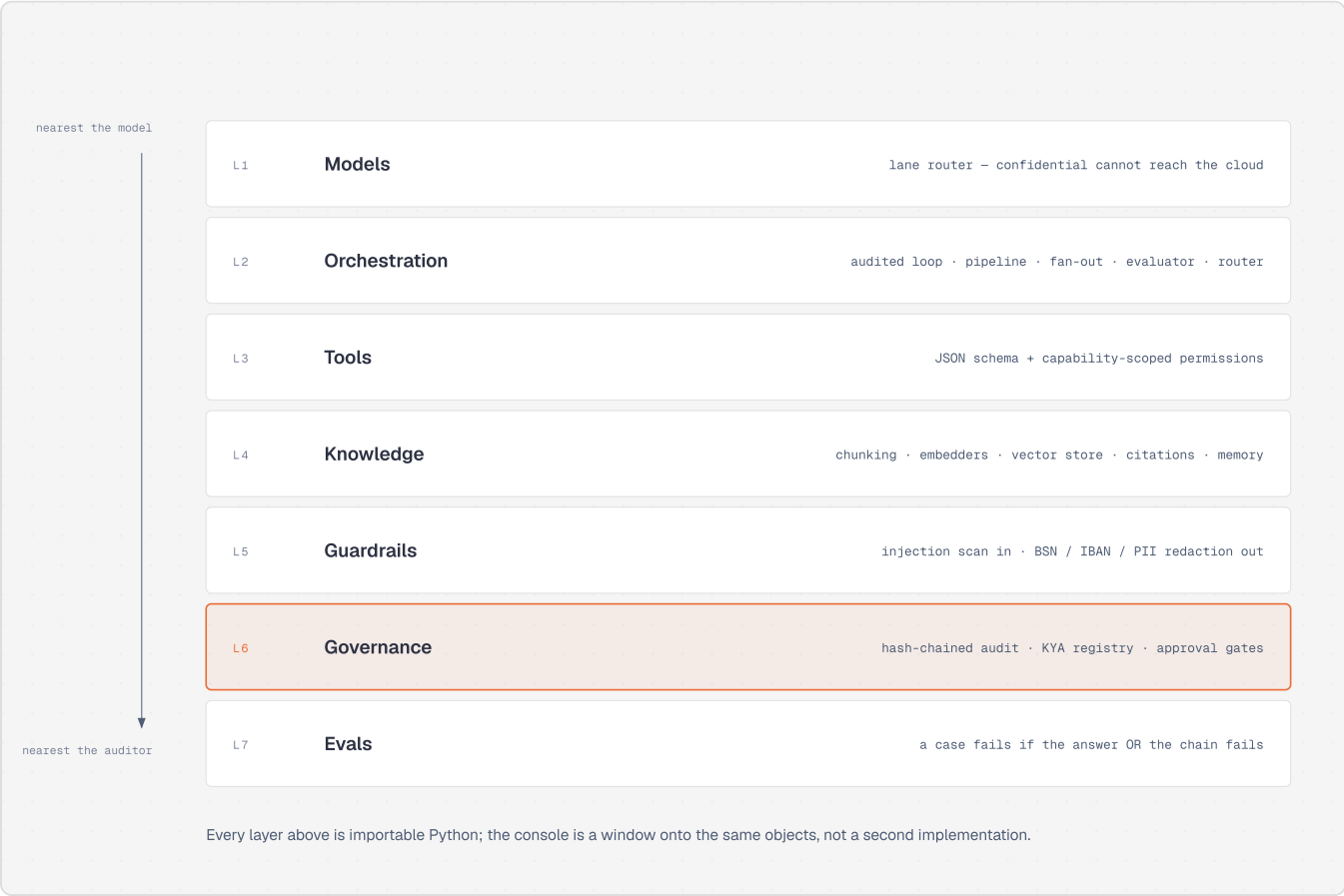

Fig. 1 — each layer importable on its own; the web console is a window onto the same objects, not a second implementation.

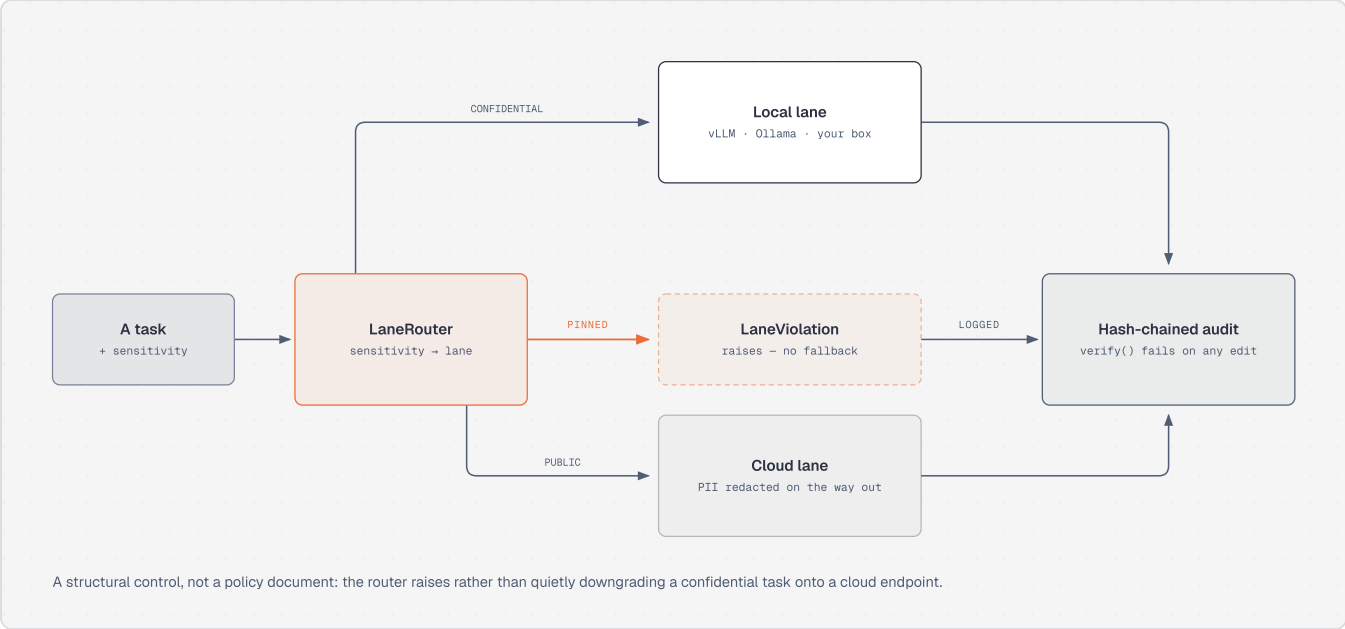

Fig. 2 — the router raises; it does not downgrade.

Most "the data stays in the EU" claims are policy documents. This one is a type error. Every step also appends to a SHA-256 hash chain, so editing, deleting or reordering a past record makes `verify()` fail. Agents act under a registered identity with a scoped capability set and instant revocation — revoke one and the next run is denied before the model is called, and a prompt-injected agent still cannot reach a tool outside its set.

Where I stand on guardrails

Prompt engineering is not a guardrail. A prompt is the right place to describe the behaviour I want, and it is the layer a model can be argued out of — by a user, by a document it retrieved, or by its own drift. It cannot enforce a permission, prevent a tool call, prove what happened, or refuse on my behalf.

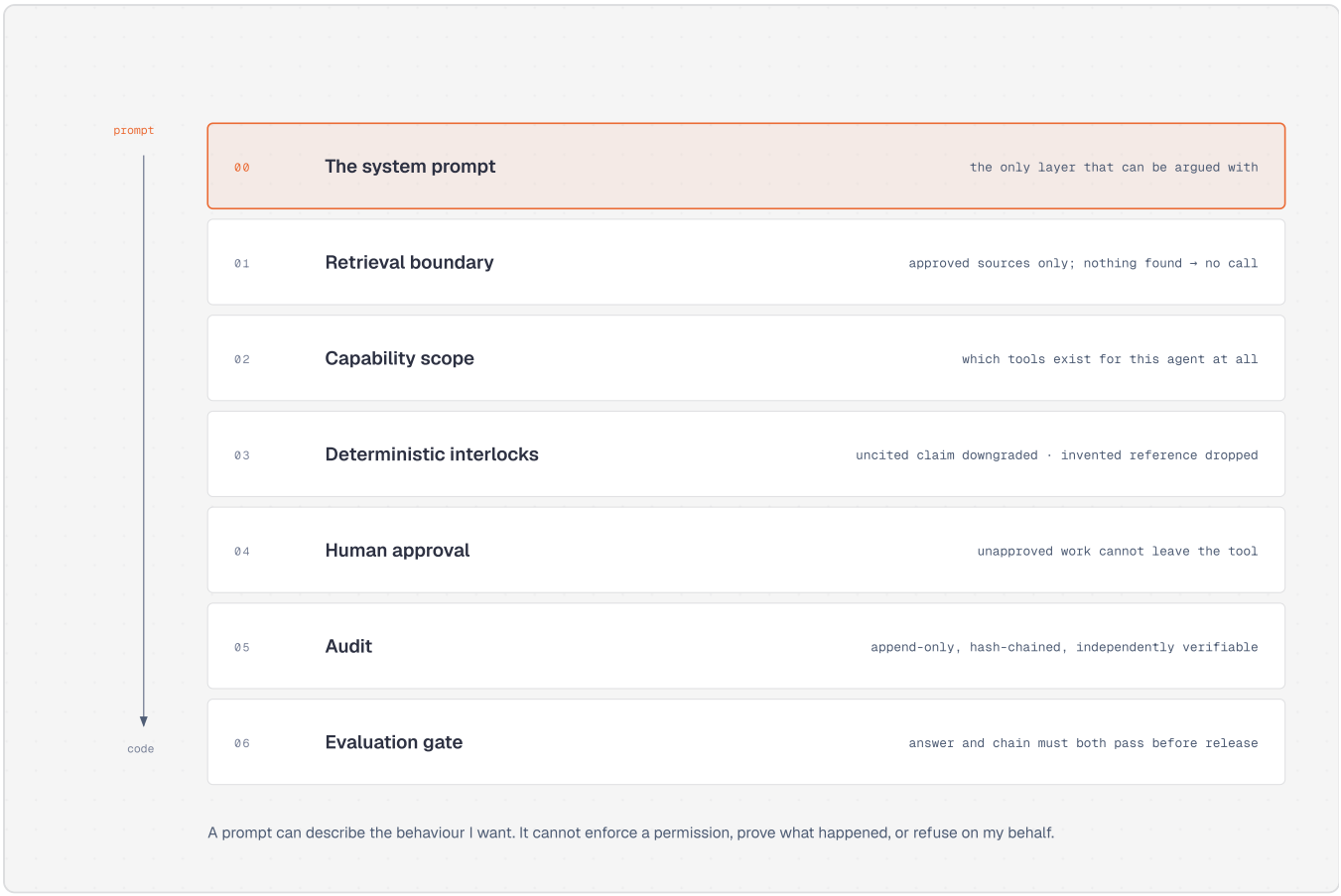

Fig. 3 — the layers that hold, and the one that does not.

> The test: if the model ignored every word of its system prompt, what would still stop it? Whatever survives that question is the guardrail. The rest is a preference.

Every interlock in the other three notes is an instance of it — the destructive-command regex that overrides a standing grant, the empty retrieval that skips the second model call, the citation outside the supplied range that is discarded, the past client name that refuses approval rather than warning. None are instructions to the model.

The stack I run on

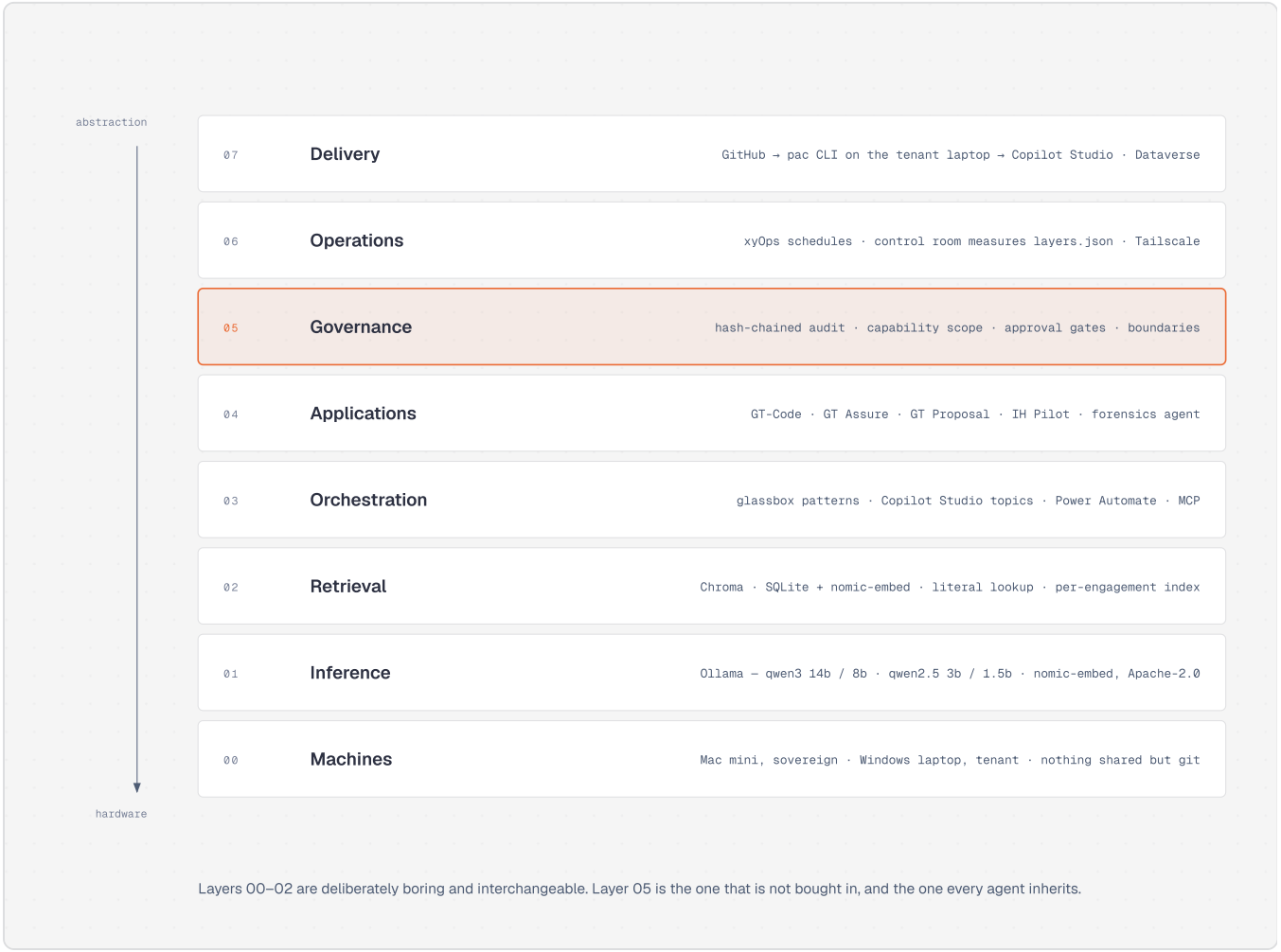

Fig. 4 — everything currently in play, by layer.

Layers 00–02 are deliberately unremarkable and interchangeable — open-weight models on Ollama, plain vector stores, no vendor I could not replace in a week. Layer 05 is the one that is not bought in, and the one every new agent inherits for free.

How an agent reaches a tenant

Deliberately boring, and shaped by one constraint: the machine I author on has no tenant access, and the machine with tenant access is not where I want an agent writing code.

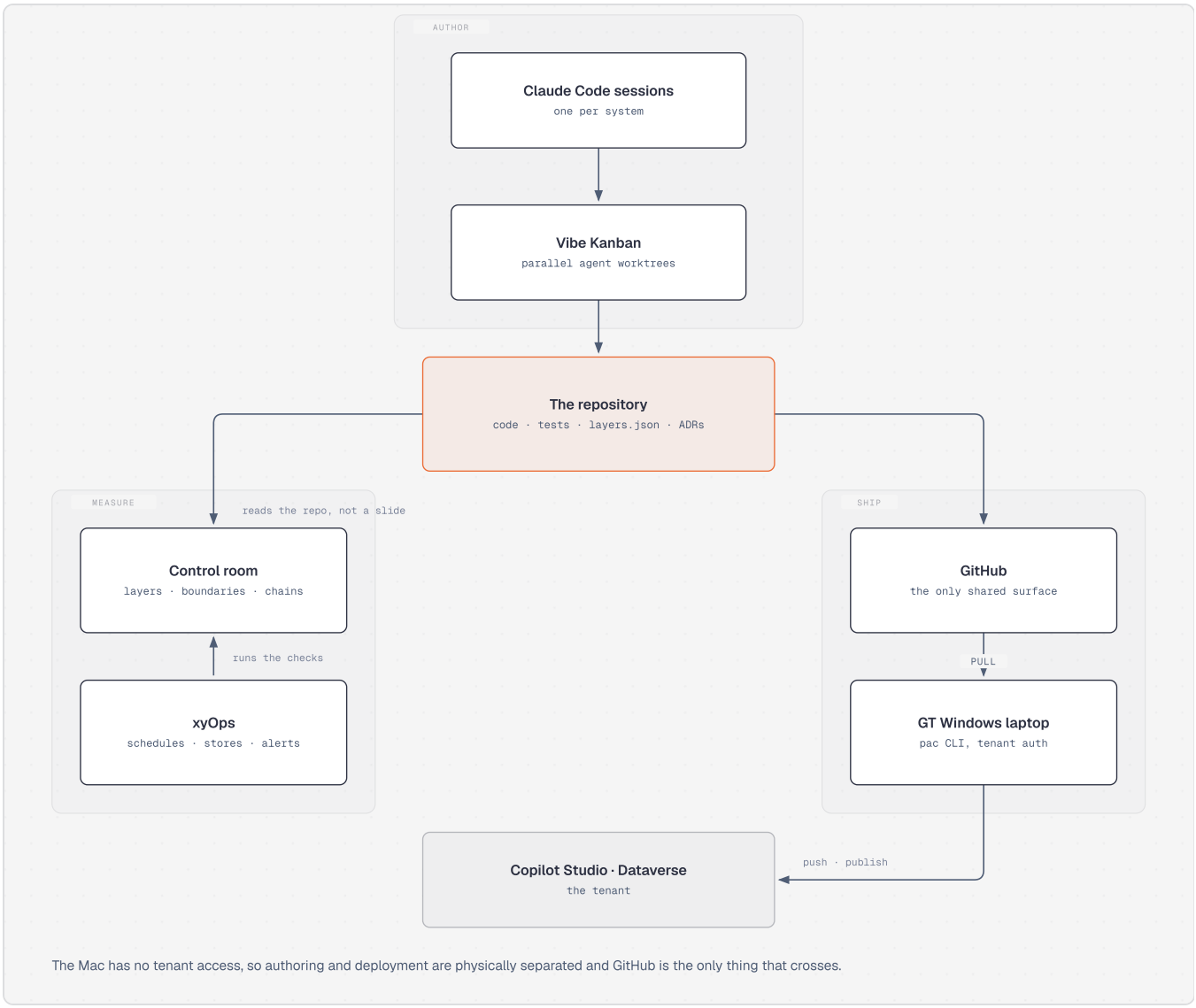

Fig. 5 — two machines, one shared surface.

STAGE	WHAT IT IS
Author	Claude Code sessions, one per system, in parallel Vibe Kanban worktrees so several agents work without sharing a working tree.
The repo	Code, tests, ADRs, and a `layers.json` declaring the system's boundary — what it reads, with which permission, where the data sits, what leaves. A new system declares its boundary before it has code.
Measure	A control room reads that file *and the code*: which layers are green, which boundaries the code actually respects, every test with its last line, the hash chains. xyOps schedules, stores and alerts.
Ship	GitHub carries the YAML to the Windows laptop, where `pac` authenticates and pushes into Copilot Studio and Dataverse.

The part I would defend hardest: the control room measures the repository, not a status slide. A boundary claim no code satisfies shows up red, and so does a test whose last line is a failure. Same principle as the interlocks, one level up.

The worked example

A forensics agent that answers questions about published Dutch fraud rulings from a curated corpus of 561 judgments — court, date, ECLI, alleged offences, forensic methods, with citations — and deliberately nothing else. Locked in YAML: `useModelKnowledge: false`, `webBrowsing: false`. Anything outside the corpus gets *"Dat staat niet in de verzamelde uitspraken."* For forensic accountants, an agent that refuses is worth more than one that improvises. The same corpus also runs as an MCP server outside Microsoft entirely — one corpus, two agents, one of which needs a tenant and a licence and one of which needs neither.

Push on this

- The layers behind the production seams — vector store, embedder, chat model, approval gate — are in-memory reference implementations. The seams are real; what is behind them is not production.
- Console engagements do not survive a restart. The audit trail does. That asymmetry is deliberate and I am not certain it is right.
- Guardrails ship pattern-based injection and PII detection. A floor, with no adversarial evaluation of where it stops working.
- The measurement plane checks a boundary is respected by the code. It cannot check the boundary was the right one.
